

Android SDK KMS Documentation 1.0

Date: 2026-01-10 00:00:00

Table of Contents

1. Introduction.....	4
2. System Requirements.....	4
3. SDK Installation.....	5
3.1 Obtaining the SDK Files.....	5
3.2 Project Configuration.....	5
3.3 Gradle Setup.....	6
3.3.1 Project-level build.gradle.kts.....	6
3.3.2 App-level build.gradle.kts.....	6
3.3.3 Gradle Properties.....	7
3.4 AndroidManifest Configuration.....	7
3.4.1 Required Permissions.....	7
3.4.2 Application Components.....	8
3.5 Accessibility Service Configuration.....	9
4. Flutter Integration.....	10
4.1 Overview.....	10
4.2 MethodChannel Setup.....	11
4.3 Flutter Service Implementation.....	12
5. SDK Initialization.....	13
5.1 Initializing the SDK.....	13
5.2 License Activation.....	14
5.3 Antivirus Engine Initialization.....	14
6. Database Updates.....	14
7. Antivirus Scanning.....	15
7.1 Scan Modes.....	15
7.2 Performing a Scan.....	16
7.3 Scan Result Structure.....	16
7.4 Threat Information.....	17
8. Real-time Protection.....	17
8.1 Starting Protection.....	17
8.2 Stopping Protection.....	18
8.3 Threat Types.....	18
9. URL Filtering.....	18
9.1 Initializing URL Filter.....	18
9.2 Requirements.....	19
10. URL Reputation.....	19
10.1 Checking URL Reputation.....	19
10.2 Extended Categories.....	20
10.3 URL Verdicts.....	20
11. Root Detection.....	20
11.1 Checking Root Status.....	21
11.2 Root Cause Information.....	21
12. Wi-Fi Reputation.....	21

12.1 Checking Network Security.....	21
12.2 Wi-Fi Verdicts.....	22
13. Compromised Accounts.....	22
13.1 Checking Email for Breaches.....	22
13.2 Breach Information.....	23
14. Compromised Passwords.....	23
14.1 Checking Password Security.....	23
15. Non-Market Apps Detection.....	23
15.1 Scanning for Non-Market Apps.....	24
15.2 Installer Sources.....	24

1. Introduction

The Kaspersky Protection SDK provides comprehensive mobile security capabilities for Android applications. This guide covers the installation and initial setup of the SDK for integration into your Android project.

The SDK includes the following core components:

Component	Description
Antivirus Engine	On-demand and real-time malware scanning
License Management	Activation and validation of SDK licenses
Database Updater	Automatic threat database updates
URL Filtering	Web protection with accessibility service
URL Reputation	Cloud-based URL reputation checking
Real-time Protection	Background monitoring service
Root Detection	Device root status detection
Wi-Fi Reputation	Network security verification
Compromised Accounts	Email breach checking
Compromised Passwords	Password breach checking
Non-Market Apps	Detection of apps from unknown sources

2. System Requirements

Requirement	Minimum Version
Android OS	6.0 (API 23) or higher
Android SDK (compileSdk)	34 or higher
Kotlin	2.0.0+
Gradle	8.0+
Java	11
NDK	27.x (recommended)

Gradle Properties Requirements:

```
android.enableJetifier=true
android.useAndroidX=true
```

3. SDK Installation

3.1 Obtaining the SDK Files

The SDK is distributed as two AAR (Android Archive) files:

File	Description	Size
<code>KL_Base_SDK_Android_5.21.0.209_Release.aar</code>	Core SDK components	~210 KB
<code>KL_Mobile_SDK_Android_5.21.0.209_FullRelease.aar</code>	Full feature set including antivirus	~6 MB

Installation Steps:

1. Create a `libs` directory inside your Android app module:

```
android/app/libs/
```

2. Copy both AAR files into the `libs` directory:

```
android/
├─ app/
│   └─ libs/
│       ├── KL_Base_SDK_Android_5.21.0.209_Release.aar
│       └─ KL_Mobile_SDK_Android_5.21.0.209_FullRelease.aar
```

Note: The file names may vary depending on the SDK version. Adjust the version numbers accordingly.

3.2 Project Configuration

The project structure should follow this layout:

```
android/
├─ app/
│   ├── libs/
│   │   ├── KL_Base_SDK_Android_5.21.0.209_Release.aar
│   │   └─ KL_Mobile_SDK_Android_5.21.0.209_FullRelease.aar
│   └─ src/
│       └─ main/
```

```

├── kotlin/
│   ├── com/
│   │   └── yourpackage/
│   │       ├── MainActivity.kt
│   │       └── repositories/
│   │           └── LicenseRepository.kt
│   └── res/
│       ├── xml/
│       │   └── accessibility_service_config.xml
│       └── AndroidManifest.xml
├── build.gradle.kts
├── build.gradle.kts
├── gradle.properties
└── settings.gradle.kts

```

3.3 Gradle Setup

3.3.1 Project-level *build.gradle.kts*

Ensure Kotlin plugin is configured in your project-level *settings.gradle.kts*:

```

plugins {
    id("com.android.application") version "8.7.3" apply false
    id("org.jetbrains.kotlin.android") version "2.1.0" apply false
}

```

3.3.2 App-level *build.gradle.kts*

Configure your app module's *build.gradle.kts*:

```

plugins {
    id("com.android.application")
    id("kotlin-android")
}

android {
    namespace = "com.yourpackage"
    compileSdk = 34
    ndkVersion = "27.3.13750724"

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }

    kotlinOptions {

```

```

        jvmTarget = JavaVersion.VERSION_11.toString()
    }

    defaultConfig {
        applicationId = "com.yourpackage.app"
        minSdk = 23
        targetSdk = 34
    }
}

dependencies {
    // Kaspersky SDK Libraries

    implementation(files("$projectDir/libs/KL_Base_SDK_Android_5.21.0.209_Release.aar"))

    implementation(files("$projectDir/libs/KL_Mobile_SDK_Android_5.21.0.209_FullRelease.aar"))

    // Required Dependencies
    implementation("com.squareup.okhttp3:okhttp:4.12.0")
    implementation("com.google.code.gson:gson:2.12.0")
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
}

```

3.3.3 Gradle Properties

Ensure your `gradle.properties` contains:

```

android.enableJetifier=true
android.useAndroidX=true
org.gradle.jvmargs=-Xmx1536M -Dkotlin.daemon.jvm.options="-Xmx1536M"

```

3.4 AndroidManifest Configuration

Add the required permissions and service declarations to your `AndroidManifest.xml`:

3.4.1 Required Permissions

```

<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <!-- Core Permissions -->
    <uses-permission android:name="android.permission.INTERNET" />

```

```

    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"
/>
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
    <uses-permission
android:name="android.permission.FOREGROUND_SERVICE_DATA_SYNC" />
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <uses-permission
android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
    <uses-permission android:name="android.permission.WAKE_LOCK" />

    <!-- Storage Permissions for Scanning -->
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
/>
    <uses-permission
android:name="android.permission.MANAGE_EXTERNAL_STORAGE" />
    <uses-permission android:name="android.permission.READ_MEDIA_VIDEO" />
    <uses-permission android:name="android.permission.READ_MEDIA_AUDIO" />
    <uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />
    <uses-permission
android:name="android.permission.WRITE_EXTERNAL_STORAGE"
    android:maxSdkVersion="29" />

    <!-- App Scanning Permission -->
    <uses-permission android:name="android.permission.QUERY_ALL_PACKAGES"
    tools:ignore="QueryAllPackagesPermission" />

    <!-- URL Filtering (Accessibility) -->
    <uses-permission
android:name="android.permission.BIND_ACCESSIBILITY_SERVICE" />

    <!-- Battery Optimization -->
    <uses-permission
android:name="android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS" />

    <!-- ... -->
</manifest>

```

3.4.2 Application Components

Inside the `<application>` tag, add the following components:

```

<application
    android:label="Your App Name"
    android:requestLegacyExternalStorage="true">

```

```

<!-- Real-time Protection Service -->
<service
    android:name=".services.RealtimeProtectionService"
    android:enabled="true"
    android:exported="false"
    android:foregroundServiceType="dataSync" />

<!-- URL Filtering Accessibility Service -->
<service
    android:name="com.morp.sdk.url_filter.MorpAccessibility"
    android:exported="true"
    android:permission="android.permission.BIND_ACCESSIBILITY_SERVICE">
    <intent-filter>
        <action
            android:name="android.accessibilityservice.AccessibilityService" />
        </intent-filter>
        <meta-data
            android:name="android.accessibilityservice"
            android:resource="@xml/accessibility_service_config" />
    </service>

<!-- Boot Receiver for Auto-start -->
<receiver
    android:name=".receivers.BootReceiver"
    android:enabled="true"
    android:exported="true"
    tools:ignore="ExportedReceiver">
    <intent-filter android:priority="1000">
        <action android:name="android.intent.action.BOOT_COMPLETED" />
        <action android:name="android.intent.action.QUICKBOOT_POWERON"
    />
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
</receiver>

</application>

```

3.5 Accessibility Service Configuration

For URL filtering functionality, create the accessibility service configuration file.

File: android/app/src/main/res/xml/accessibility_service_config.xml

```

<accessibility-service
    xmlns:android="http://schemas.android.com/apk/res/android"

```

```

android:accessibilityEventTypes="typeWindowContentChanged|typeWindowStateCh
anged|typeViewFocused"
    android:accessibilityFeedbackType="feedbackGeneric"

android:accessibilityFlags="flagReportViewIds|flagRetrieveInteractiveWindow
s"
    android:canRetrieveWindowContent="true"
    android:description="@string/accessibility_service_description"
    android:notificationTimeout="100"

android:packageNames="com.android.chrome,org.mozilla.firefox,com.opera.brow
ser,com.microsoft.emmx,com.UCMobile.intl,com.brave.browser,com.yandex.brows
er,com.sec.android.app.sbrowser" />

```

Add the description string to your `strings.xml`:

```

<string name="accessibility_service_description">
    This service monitors browser URLs to protect you from malicious
websites.
</string>

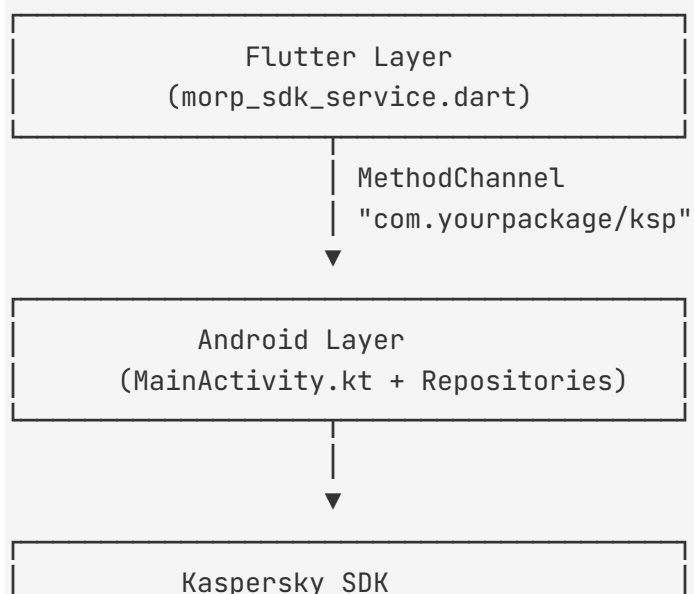
```

4. Flutter Integration

4.1 Overview

The SDK integrates with Flutter through Platform Channels, specifically using [MethodChannel](#) for communication between Dart and the Kotlin implementation.

Architecture:



4.2 MethodChannel Setup

Configure the MethodChannel in your `MainActivity.kt`:

```
import io.flutter.embedding.android.FlutterActivity
import io.flutter.embedding.engine.FlutterEngine
import io.flutter.plugin.common.MethodChannel

class MainActivity : FlutterActivity() {
    private val CHANNEL_NAME = "com.yourpackage/ksp"

    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)

        val channel = MethodChannel(
            flutterEngine.dartExecutor.binaryMessenger,
            CHANNEL_NAME
        )

        channel.setMethodCallHandler { call, result →
            when (call.method) {
                "activate_license" → {
                    val licenseKey = call.argument<String>("license_key")
                    // Call SDK initialization (run on background thread)
                    // Return result.success(true) or result.error(...)
                }
                "is_license_activated" → {
                    // Check license status
                    // Return result.success(isValid)
                }
                "update_db" → {
                    // Update database (run on background thread)
                    // Return result.success(true) or result.error(...)
                }
                else → result.notImplemented()
            }
        }
    }
}
```

4.3 Flutter Service Implementation

Create a service class in Dart to interact with the SDK:

```
import 'package:flutter/services.dart';

class KspSdkService {
  static const _channel = MethodChannel('com.yourpackage/ksp');

  /// Activates the SDK license
  static Future<void> activateLicense(String licenseKey) async {
    await _channel.invokeMethod('activate_license', {
      'license_key': licenseKey,
    });
  }

  /// Checks if the license is currently activated
  static Future<bool> isLicenseActivated() async {
    final result = await _channel.invokeMethod('is_license_activated') as bool;
    return result;
  }

  /// Updates the antivirus database
  static Future<bool> updateDatabase() async {
    final result = await _channel.invokeMethod('update_db') as bool;
    return result;
  }
}
```

Usage in Flutter:

```
// Initialize and activate during app startup
await KspSdkService.activateLicense('YOUR-LICENSE-KEY');

// Check activation status
final isActivated = await KspSdkService.isLicenseActivated();
print('License activated: $isActivated');

// Update threat databases
await KspSdkService.updateDatabase();
```

5. SDK Initialization

5.1 Initializing the SDK

The SDK must be initialized before any other operations.

Note: SDK initialization and license activation perform network operations. Run these on a background thread (e.g., using Kotlin Coroutines or ExecutorService).

```
import android.content.Context
import com.kavsdk.KavSdk
import com.kavsdk.antivirus.AntivirusInstance
import java.io.File

fun initializeSdk(context: Context, licenseKey: String) {
    // Step 1: Create bases directory for threat databases
    val basesDir = File(context.filesDir, "kav_bases")
    basesDir.mkdirs()

    // Step 2: Initialize SDK
    if (!KavSdk.isInitialized()) {
        KavSdk.initSafe(context, basesDir, null as String?)
    }

    // Step 3: Activate license
    val license = KavSdk.getLicense()
    license.activate(licenseKey)

    // Step 4: Initialize Antivirus engine
    val antivirus = AntivirusInstance.getInstance()
    val scanTmpDir = File(context.cacheDir, "scan_tmp")
    val monitorTmpDir = File(context.cacheDir, "monitor_tmp")
    scanTmpDir.mkdirs()
    monitorTmpDir.mkdirs()

    antivirus.initAntivirus(
        context,
        scanTmpDir.absolutePath,
        monitorTmpDir.absolutePath
    )
}

fun isLicenseActivated(): Boolean {
    if (!KavSdk.isInitialized()) {
        return false
    }
}
```

```

    val license = KavSdk.getLicense()
    return license.isValid
}

```

5.2 License Activation

The license activation process involves:

Step	Method	Description
1	<code>KavSdk.initSafe()</code>	Initialize the SDK with context and bases directory
2	<code>KavSdk.getLicense()</code>	Obtain the license object
3	<code>license.activate()</code>	Activate with the license key

Important Notes:

- Always run initialization on a background thread (IO dispatcher)
- Check `KavSdk.isInitialized()` before initializing to prevent duplicate initialization
- Store the license key securely; do not hardcode in production applications

5.3 Antivirus Engine Initialization

After license activation, initialize the antivirus engine:

```

val antivirus = AntivirusInstance.getInstance()

// Create temporary directories for scanning operations
val scanTmpDir = File(context.cacheDir, "scan_tmp")
val monitorTmpDir = File(context.cacheDir, "monitor_tmp")
scanTmpDir.mkdirs()
monitorTmpDir.mkdirs()

antivirus.initAntivirus(
    context,
    scanTmpDir.absolutePath,
    monitorTmpDir.absolutePath
)

```

6. Database Updates

Threat databases must be updated regularly to ensure protection against the latest threats.

Note: Database updates perform network operations. Run on a background thread.

```
import com.kavsdk.updater.Updater
import com.kavsdk.updater.UpdateEventListener

fun updateDatabase() {
    val updater = Updater.getInstance()

    val listener = object : UpdateEventListener {
        override fun onUpdateEvent(eventType: Int, eventResult: Int):
Boolean {
            // Return true to cancel update, false to continue
            return false
        }
    }

    updater.updateAllBases(listener)
}
```

Update Frequency Recommendations:

Update Type	Frequency
On app launch	Always check for updates
Background updates	Every 4-6 hours
Manual refresh	On user request

7. Antivirus Scanning

The SDK provides on-demand scanning with different scan modes.

7.1 Scan Modes

Mode	Description
Basic	Only installed applications. Uses KSN cloud only.
Light	Includes Basic + root detection + Downloads directory.
LightPlus	Includes Light + uses local threat databases.
Recommended	Includes LightPlus + entire internal storage.

Mode	Description
Full	Includes Recommended + external storage + adware detection.

7.2 Performing a Scan

Note: Scanning operations can take time. Run on a background thread.

```
import com.kavsdk.antivirus.AntivirusInstance
import com.kavsdk.antivirus.ThreatInfo
import com.kavsdk.antivirus.easyscanner.EasyMode

fun performScan(mode: String): Map<String, Any> {
    val antivirus = AntivirusInstance.getInstance()
    val scanner = antivirus.createEasyScanner()

    val easyMode = when (mode.lowercase()) {
        "basic" → EasyMode.Basic
        "light" → EasyMode.Light
        "lightplus" → EasyMode.LightPlus
        "full" → EasyMode.Full
        else → EasyMode.Recommended
    }

    val scanResult = scanner.scan(easyMode)

    return mapOf(
        "filesScanned" to scanResult.filesScanned,
        "objectsScanned" to scanResult.objectsScanned,
        "isRooted" to scanResult.isRooted,
        "malwareCount" to scanResult.malwareList.size,
        "riskwareCount" to scanResult.riskwareList.size
    )
}
```

7.3 Scan Result Structure

Field	Type	Description
filesScanned	Int	Number of files scanned
objectsScanned	Int	Number of objects scanned
isRooted	Boolean	Device root status

Field	Type	Description
malwareList	List	Detected malware threats
riskwareList	List	Detected riskware

7.4 Threat Information

Each detected threat contains:

Field	Description
virusName	Name of the detected threat
fileFullPath	Full path to the infected file
packageName	Package name (for applications)
severityLevel	Threat severity level
categories	Threat categories

8. Real-time Protection

Real-time protection monitors the device for threats in the background.

8.1 Starting Protection

```
import com.kavsdk.antivirus.AntivirusInstance
import com.kavsdk.antivirus.MonitorEventListener
import com.kavsdk.antivirus.ThreatInfo
import com.kavsdk.antivirus.ThreatType

fun startRealtimeProtection() {
    val antivirus = AntivirusInstance.getInstance()

    antivirus.setMonitorListener(object : MonitorEventListener {
        override fun onMonitorEvent(threatInfo: ThreatInfo, threatType: ThreatType) {
            // Handle threat detection
            val virusName = threatInfo.virusName
            val filePath = threatInfo.fileFullPath
            // Notify user or take action
        }
    })

    antivirus.setMonitorState(true)
```

```
}
```

8.2 Stopping Protection

```
fun stopRealtimeProtection() {  
    val antivirus = AntivirusInstance.getInstance()  
    antivirus.setMonitorState(false)  
    antivirus.setMonitorListener(null)  
}
```

8.3 Threat Types

Type	Description
Malware	Malicious software
Riskware	Potentially unwanted application
Adware	Advertising software

9. URL Filtering

URL filtering protects users from malicious websites using the accessibility service.

9.1 Initializing URL Filter

```
import android.content.Context  
import com.kavsdk.webfilter.WebAccessEvent  
import com.kavsdk.webfilter.WebAccessHandler  
import com.kavsdk.webfilter.WebFilterControl  
import com.kavsdk.webfilter.WebFilterControlFactoryImpl  
  
object UrlFilterManager {  
    private var webFilterControl: WebFilterControl? = null  
  
    fun init(context: Context) {  
        val handler = object : WebAccessHandler {  
            override fun onWebAccess(event: WebAccessEvent) {  
                // Handle web access events  
            }  
        }  
  
        val factory = WebFilterControlFactoryImpl()  
        webFilterControl = factory.create(handler,  
context.applicationContext)  
    }  
}
```

```

fun setEnabled(enabled: Boolean) {
    webFilterControl?.enable(enabled)
}

fun isEnabled(): Boolean {
    return webFilterControl?.isEnabled ?: false
}
}

```

Usage:

```

// Initialize (once, e.g., in Application or Activity)
UrlFilterManager.init(context)

// Enable/disable filtering
UrlFilterManager.setEnabled(true)

// Check status
val isActive = UrlFilterManager.isEnabled()

```

9.2 Requirements

- Accessibility service must be enabled by the user
- Service declaration in AndroidManifest.xml
- Accessibility configuration file

10. URL Reputation

Check URL reputation using Kaspersky Security Network (KSN).

10.1 Checking URL Reputation

Note: This operation requires network access. Run on a background thread.

```

import android.content.Context
import com.kaspersky.components.urlchecker.UrlCategory
import com.kaspersky.components.urlchecker.UrlInfo
import com.kavsdk.urlchecker.UrlCheckService

fun checkUrlReputation(context: Context, url: String): Map<String, Any> {
    val service = UrlCheckService(context.applicationContext)
    val info = service.checkUrl(url)

    val verdict = when (info?.mVerdict) {
        UrlInfo.VERDICT_GOOD → "safe"
        UrlInfo.VERDICT_BAD → "dangerous"
    }
}

```

```

        else → "unknown"
    }

    val categories = info?.mCategories?.let {
        UrlCategory.getCategoriesByMask(it).map { cat → cat.name }
    } ?: emptyList()

    return mapOf(
        "verdict" to verdict,
        "categories" to categories
    )
}

```

10.2 Extended Categories

For extended category information:

```

fun checkUrlReputationExtended(context: Context, url: String): Map<String, Any> {
    val service = UrlCheckService(context.applicationContext)
    val info = service.checkUrlExt(url)

    val categories = info?.mCategoriesExt?.map { it.name } ?: emptyList()

    return mapOf(
        "verdict" to (info?.mVerdict ?: -1),
        "extendedCategories" to categories
    )
}

```

10.3 URL Verdicts

Verdict	Description
VERDICT_GOOD	URL is safe
VERDICT_BAD	URL is malicious
VERDICT_UNKNOWN	Unknown reputation

11. Root Detection

Detect if the device has been rooted.

11.1 Checking Root Status

```
import com.kavsdk.rootdetector.RootCause
import com.kavsdk.rootdetector.RootDetector

fun checkRootStatus(): Map<String, Any?> {
    val detector = RootDetector.getInstance()
    val isRooted = detector.checkRoot()

    val rootCause = if (isRooted) detector.rootCause else null

    return mapOf(
        "isRooted" to isRooted,
        "rootCausePath" to rootCause?.rootCausePath,
        "selfCheckFailed" to (rootCause?.isSelfCheckFailed ?: false)
    )
}
```

11.2 Root Cause Information

Field	Description
rootCausePath	Path that indicates root access
isSelfCheckFailed	SDK integrity check status

12. Wi-Fi Reputation

Check the security of the current Wi-Fi network.

12.1 Checking Network Security

Note: Requires `ACCESS_WIFI_STATE` permission.

```
import android.content.Context
import com.kavsdk.wifi.Verdict
import com.kavsdk.wifi.WifiReputation

fun checkWifiSecurity(context: Context): Map<String, Any> {
    val wifiReputation = WifiReputation(context.applicationContext)
    val result = wifiReputation.checkCurrentNetwork()

    return mapOf(
        "ssid" to result.ssid,
        "bssid" to result.bssid,
        "verdict" to result.verdict.name,
    )
}
```

```

        "isSafe" to (result.verdict == Verdict.Safe)
    )
}

```

12.2 Wi-Fi Verdicts

Verdict	Description
Safe	Network is considered safe
Unsafe	Network may be compromised
Unknown	Unable to determine security

13. Compromised Accounts

Check if an email address has been involved in data breaches.

13.1 Checking Email for Breaches

Note: This is a cloud-based check. Run on a background thread.

```

import com.kavsdk.compromised_accounts.CompromisedAccounts

fun checkCompromisedAccount(email: String): Map<String, Any?> {
    val response = CompromisedAccounts.getBreaches(email)

    val breaches = response?.breaches?.map { breach →
        mapOf(
            "name" to breach.name,
            "domain" to breach.domain,
            "pwnCount" to breach.pwnCount,
            "date" to breach.date,
            "dataClasses" to breach.dataClasses?.map { it.name }
        )
    } ?: emptyList()

    return mapOf(
        "errorCode" to (response?.errorCode ?: -1),
        "breaches" to breaches
    )
}

```

13.2 Breach Information

Field	Description
<code>name</code>	Name of the breached service
<code>domain</code>	Domain of the breached service
<code>pwnCount</code>	Number of accounts affected
<code>date</code>	Date of the breach
<code>dataClasses</code>	Types of data exposed

14. Compromised Passwords

Check if a password has been exposed in data breaches.

14.1 Checking Password Security

Note: Passwords are hashed before checking. Run on a background thread.

```
import com.kavsdk.compromised_passwords.CompromisedPasswords

fun checkCompromisedPassword(password: String): Map<String, Any?> {
    val response = CompromisedPasswords.getBreaches(password)

    val breaches = response?.breaches?.map { breach →
        mapOf(
            "name" to breach.name,
            "domain" to breach.domain,
            "date" to breach.date
        )
    } ?: emptyList()

    return mapOf(
        "errorCode" to (response?.errorCode ?: -1),
        "breachCount" to breaches.size,
        "breaches" to breaches
    )
}
```

15. Non-Market Apps Detection

Detect applications installed from sources other than official app stores.

15.1 Scanning for Non-Market Apps

```
import android.content.Context
import com.kavsdk.nonmarketapps.InstalledApplications

fun scanNonMarketApps(context: Context): List<Map<String, String>> {
    val installedApps = InstalledApplications(context.applicationContext)
    val nonMarketApps = installedApps.getNonMarketApplications(
        installedApps.GOOGLE_PLAY_ONLY
    )

    val pm = context.packageManager

    return nonMarketApps.map { appInfo →
        val androidInfo = appInfo.applicationInfo
        mapOf(
            "packageName" to androidInfo.packageName,
            "applicationName" to androidInfo.loadLabel(pm).toString(),
            "installerPackageName" to (appInfo.installer ?: "unknown")
        )
    }
}
```

15.2 Installer Sources

Constant	Description
GOOGLE_PLAY_ONLY	Only Google Play is trusted
ALL_MARKETS	All known markets are trusted